ENSF 338 — Data Structures and Algorithms

Term Project: Algorithm Design Report

Campus Navigation and Event Management System

Department of Electrical and Software Engineering
University of Calgary — Winter 2026

Course: ENSF 338
Project: Campus Navigation & Event Management System
Term: Winter 2026

Requirements Coverage

Requirement (Section 2)	Specification	Report Section
2.1 Campus Map & Shortest Path	Graph + A*	Shortest Path Navigation
2.2 Route History & Undo	LIFO ring buffer	Route History
2.3 Room & Event Booking	Ring buffer + AVL archive	Room and Event Booking System
2.4 Service Queue System	Three-tier deque	Service Request Processing
2.5 Fast Building Lookup	Hash map / AVL	Campus Data Structure
2.6 Incoming Request Processing	FIFO deque	Request Pipeline
2.7 Bonus: Balanced Event Index	Self-balancing BST (AVL)	AVL Tree

Contents

1 Campus Data Structure **3**
1.1 Overview . . . 3
1.2 Class Hierarchy . . . 3
1.3 Design Decisions . . . 4
1.3.1 Hash Table for Building Lookup (`Campus.buildings`) . . . 4
1.3.2 List for Floors and Rooms . . . 4
1.3.3 Dual Representation: Building Node in the Graph . . . 5
1.4 Complexity Analysis . . . 5
1.5 Summary . . . 5

2 Priority Service Pipeline (2.4) **6**
2.1 Overview . . . 6
2.2 Design Decision: Three Deques over a Single Heap Priority Queue . . . 6
2.3 Priority-Ordered Dequeue . . . 6
2.4 Complexity Analysis . . . 6
2.5 Summary . . . 7

3 Campus Map and Shortest Path Navigation (2.1) **8**
3.1 Overview . . . 8
3.2 Node Structure . . . 8
3.3 Edge Weights: Euclidean Distance as the Cost Function . . . 8
3.4 Pathfinding Algorithm: A* . . . 9
3.5 Complexity Analysis . . . 10
3.6 Summary . . . 10

4 Route History and Navigation Undo (2.2) **11**
4.1 Overview . . . 11
4.2 Design Decision: LIFO Ring Buffer . . . 11
4.3 Complexity Analysis . . . 11

5 Room and Event Booking System (2.3, 2.5) **12**
5.1 Overview . . . 12
5.2 Class Hierarchy and Design Decisions . . . 12
5.2.1 DailyBooking — Flat List of Time Slots . . . 12
5.2.2 BookingSystem — LIFO Ring Buffer as a Rolling Day Window . . . 13
5.3 Complexity Analysis . . . 13
5.4 Summary . . . 14

6 Incoming Request Processing Pipeline (2.6) **15**
6.1 Overview . . . 15
6.2 Request Wrapper . . . 15
6.3 Design Decision: Deque as the Underlying Structure . . . 16
6.4 Request Execution Model . . . 16

6.5 Complexity Analysis 16
6.6 Summary 16

7 AVL Tree — Balanced Event Index (2.5, 2.7) 17
7.1 Overview 17
7.2 Complexity Analysis 17
7.3 Summary 17

8 Contributions 18

9 Ai Use Declaration 18

Campus Data Structure

Overview

The `Campus` class serves as the central data structure of the entire system, responsible for initialising and owning every other component: the physical model of the university (buildings, floors, rooms, pathways), the navigation graph, and the service lookup table.

Class Hierarchy

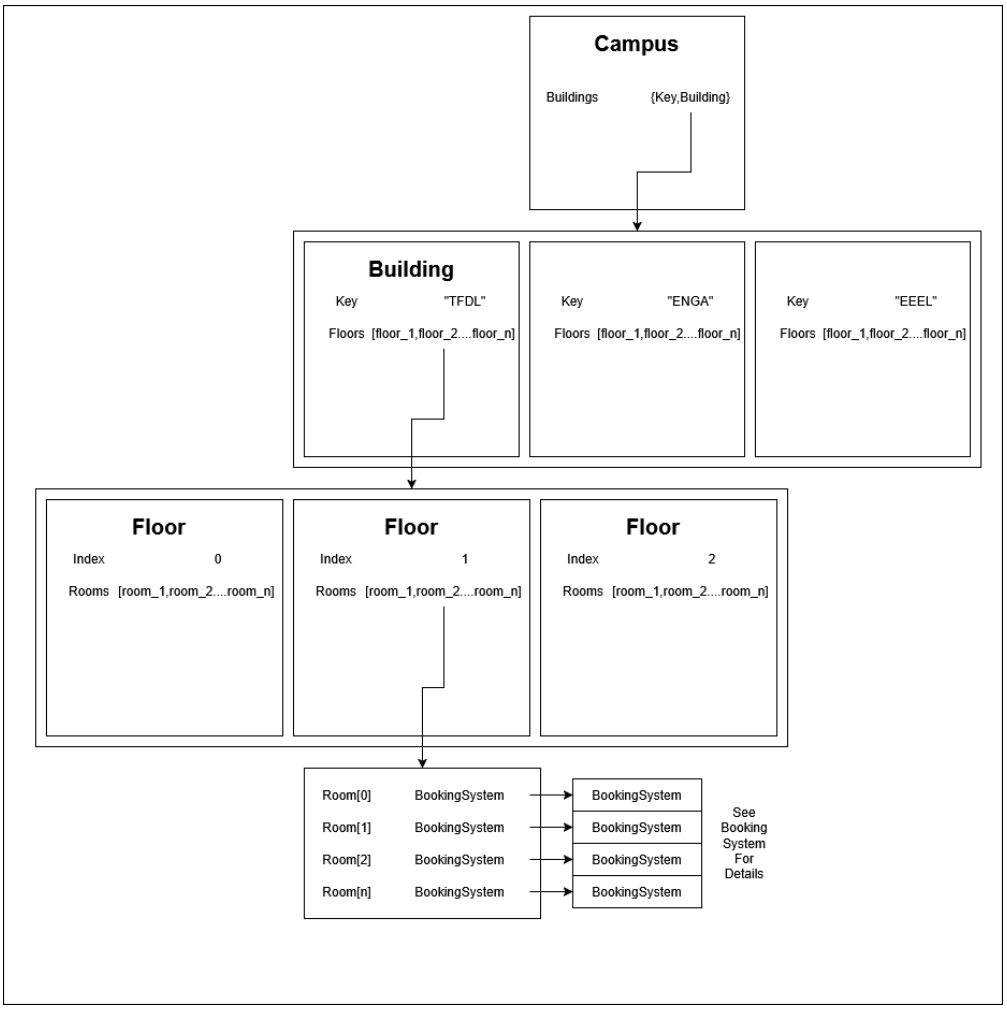

Figure 1: DataModel layout.

The physical campus is modelled through a four-level hierarchy of plain Python classes.

Class	Key Fields	Role
`Location`	`x_position`, `y_position`	Lightweight value object storing 2-D grid coordinates.
`Room`	`id`, `room_type`, `booking`	Leaf node of the hierarchy; owns a `BookingSystem` instance.
`Floor`	`id`, `rooms`	Ordered list of `Room` objects on a single storey.
`Building`	`bid`, `name`, `node`, `floors`	Represents one campus building; holds a graph `Node` for navigation.
`Pathway`	`node`	An intermediate waypoint on the campus; contributes a `Node` to the graph.
`Campus`	`buildings`, `pathways`, `campus_graph`, ...	Aggregate root that owns every other object.

Design Decisions

Hash Table for Building Lookup (`Campus.buildings`)

Buildings are stored in a custom `HashTable` keyed by unique building ID string (e.g. `"ICT"`, `"TFDL"`).

```
self.buildings = hash_table.HashTable()          # building_id (str) -> Building
self.buildings[bid] = new_building
```

Listing 1: Building dictionary in Campus

Why a hash table? The dominant operation on the building store is *given a building ID, return the Building object.* A hash table gives $O(1)$ average-case lookup, insertion, and deletion. With 46 UCalgary buildings initialised at startup and more potentially added at runtime, this scales well. Collision resolution is handled internally; in practice, string keys achieve $O(1)$ with a suitable hash function.

Alternative considered — list with linear search. Storing buildings in a plain list would give $O(n)$ lookup, worsening linearly as the campus grows.

List for Floors and Rooms

Floors and rooms are stored as Python `list` objects and accessed by integer index.

```
class Floor:
    def __init__(self, id):
        self.rooms = []       # index-based access: rooms[i]
        self.id = id
class Building:
    def __init__(self, ...):
        self.floors = []      # index-based access: floors[i]
```

Listing 2: Floor and room storage

Why a list? Floor and room IDs are sequential integers starting at zero, matching real-world numbering. Lists give $O(1)$ index access and $O(1)$ append during initialisation. Mid-array insertion (e.g. a new floor added between existing ones) is a physical construction event and effectively never occurs at runtime, so the $O(n)$ cost of such an operation is inconsequential.

Alternative considered — dictionary keyed by floor/room ID. A dict would handle non-sequential IDs but adds unnecessary overhead when IDs are simply integers starting at zero.

Dual Representation: Building Node in the Graph

Each `Building` object stores a reference to a `tv.Node` at construction, coupling the physical model to the navigation graph without a separate lookup step.

```python
class Building:
    def __init__(self, name, bid, location: Location):
        ...
        self.node = tv.Node(bid, location.x_position, location.
            y_position)
```

Listing 3: Node attached to each Building

Complexity Analysis

Operation	Best	Average	Worst
`add_building()`	$O(f \cdot r)$	$O(f \cdot r)$	$O(f \cdot r)$
`remove_building()`	$O(1)$	$O(1)$	$O(1)$
Building lookup by ID	$O(1)$	$O(1)$	$O(n)^*$
Service lookup by name	$O(1)$	$O(1)$	$O(n)^*$
`get_rooms()` by index	$O(1)$	$O(1)$	$O(1)$

* Worst case occurs only under hash collisions; in practice, $O(1)$ is achieved for string keys.

Space complexity. $O(b \cdot f \cdot r)$ where b is the number of buildings, f the average floors, and r the average rooms per floor.

Summary

The `Campus` class uses hash tables for all keyed lookups to guarantee $O(1)$ average-case retrieval, and integer-indexed lists for floors and rooms where sequential index access is the dominant pattern.

Priority Service Pipeline (2.4)

*Fulfils requirement: **2.4** (Priority-Based Service Queue).*

Overview

The priority service pipeline maintains three independent `RequestPipeline` instances — `service_pipeline_high`, `service_pipeline_med`, and `service_pipeline_low` — one per priority tier. Each request carries an integer priority (1 = Emergency, 2 = Standard, 3 = Low). Priority ordering is enforced at dequeue time by always draining the highest non-empty tier first.

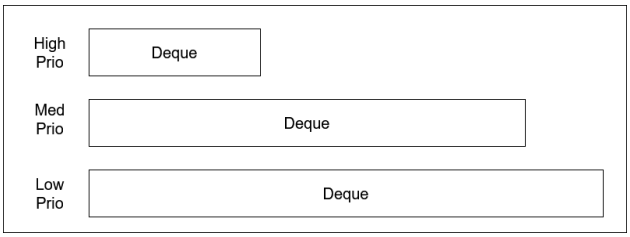

Figure 2: Priority Requests visualization.

Design Decision: Three Deques over a Single Heap Priority Queue

Each sub-pipeline is an independent `RequestPipeline` backed by a deque, giving $O(1)$ enqueue and $O(1)$ dequeue per tier. Requests within a tier are served in strict FIFO order, which a heap cannot guarantee for equal-priority items. The three-deque model also allows the GUI to display the complete processing order across all tiers, not just the next item.

Alternative considered — single heap-based priority queue. A heap holding `(priority, request)` gives $O(\log n)$ enqueue and does not preserve FIFO ordering within a priority tier. For a fixed three-tier system the three-deque approach gives strictly better complexity and equal-priority fairness.

Priority-Ordered Dequeue

`process_next_closure()` checks the high-priority pipeline first, falling through to lower tiers only when higher tiers are empty. `get_len()` is $O(1)$ on the underlying deque, so the priority check adds only three integer comparisons of negligible overhead.

Complexity Analysis

Operation	Best	Average	Worst
`enque_request()` (per tier)	$O(1)$	$O(1)$	$O(1)$
`process_next_closure()`	$O(1)$	$O(1)$	$O(1)$
`process_all()`	$O(n)$	$O(n)$	$O(n)$

n = total pending requests across all three pipelines. Space complexity is $O(n)$; the three-pipeline split adds only two extra deque headers of $O(1)$ overhead.

Summary

The priority service pipeline preserves $O(1)$ enqueue and dequeue while enforcing strict Emergency > Standard > Low ordering and FIFO fairness within each tier.

Campus Map and Shortest Path Navigation (2.1)

*Fulfils requirement: **2.1** (Campus Map and Shortest Path Navigation).*

Overview

The navigation system is implemented in `NavigationSystem/traversal.py` and composed of two core components: a `Node` class representing any campus point, and a `graph` class that owns all nodes, manages connections, and runs A* search.

Node Structure

```python
class Node:
    def __init__(self, name, x, y):
        self.x = x
        self.y = y
        self.name = name
        self.linked_nodes = []
```

Listing 4: Node class definition

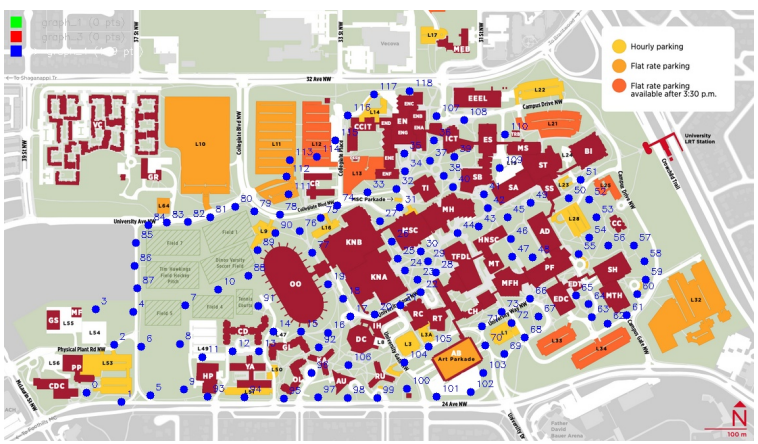

Figure 3: Pathways Node.

Why a flat adjacency list on each node? Each node owns its neighbour references (`linked_nodes`) rather than the graph maintaining a global structure. For this sparse campus graph, this is far more memory-efficient than an adjacency matrix ($O(V)$ per node vs $O(V^2)$ total).

Alternative considered — adjacency matrix. With 165 nodes, a matrix would require $165^2 = 27{,}225$ entries, the vast majority zero. The adjacency-list approach uses memory proportional only to actual edges.

Edge Weights: Euclidean Distance as the Cost Function

Edge costs are computed on-the-fly rather than stored explicitly:

```python
def get_node_vector(origin_node: Node, target_node: Node):
    distance = np.sqrt(
        (origin_node.x - target_node.x)**2 +
        (origin_node.y - target_node.y)**2
    )
    return distance
```

Listing 5: Edge cost function

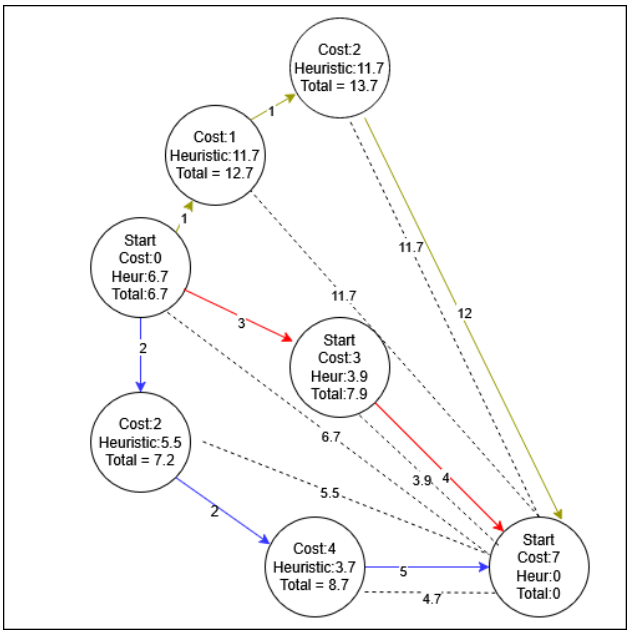

Figure 4: A* visualisation.

Node coordinates are derived directly from the campus map image, so Euclidean distance is proportional to real walking distance. This value serves simultaneously as the actual edge cost g and the heuristic h. Because straight-line distance never overestimates the true path cost, the heuristic is admissible and A* is guaranteed to return an optimal solution.

Pathfinding Algorithm: A*

Algorithm	Time Complexity	Uses Heuristic?	Optimal?
Breadth-First Search	$O(V + E)$	No	Only for unweighted graphs
Dijkstra's Algorithm	$O((V + E) \log V)$	No	Yes
A* Search	$O((V + E) \log V)$	Yes	Yes (with admissible h)

A* was chosen because it is both optimal and informed: the Euclidean heuristic focuses the search toward the destination and typically expands far fewer nodes than Dijkstra's on geometric graphs.

- `g_score` — lowest known cost from the start node, initialised to ∞.
- `came_from` — predecessor map used to reconstruct the path.

- `open_set` — custom min-heap (`PriorityHeap`) storing $(f, \text{counter}, \text{node})$ tuples; the counter breaks f-score ties.

Complexity Analysis

Operation	Best	Average	Worst
`append_node()`	$O(1)$	$O(1)$	$O(1)$
`get_node_id()`	$O(1)$	$O(1)$	$O(n)$
`node_linker()` (init only)	$O(k)$	$O(k)$	$O(k)$
`get_node_vector()`	$O(1)$	$O(1)$	$O(1)$
`find_path()` — A*	$O(1)$	$O((V+E)\log V)$	$O((V+E)\log V)$
Path reconstruction	$O(P)$	$O(P)$	$O(P)$

Space complexity. $O(V+E)$ for nodes and adjacency lists; A* adds $O(V)$ for `g_score`, `came_from`, and the open set, giving $O(V+E)$ total.

Summary

The navigation system uses an adjacency-list graph with on-the-fly Euclidean edge weights. A* combines Dijkstra's optimality with a spatial heuristic that reduces node expansions on geometric graphs. A tie-breaking counter prevents heap comparison errors when f-scores collide.

Route History and Navigation Undo (2.2)

*Fulfils requirement: **2.2** (Route History and Navigation Undo).*

Overview

Navigation undo is implemented on the `graph` class. Each navigation action is stored as a callable in a fixed-capacity LIFO ring buffer. Calling `undo()` pops and executes the most recent callable, restoring the previous navigation state.

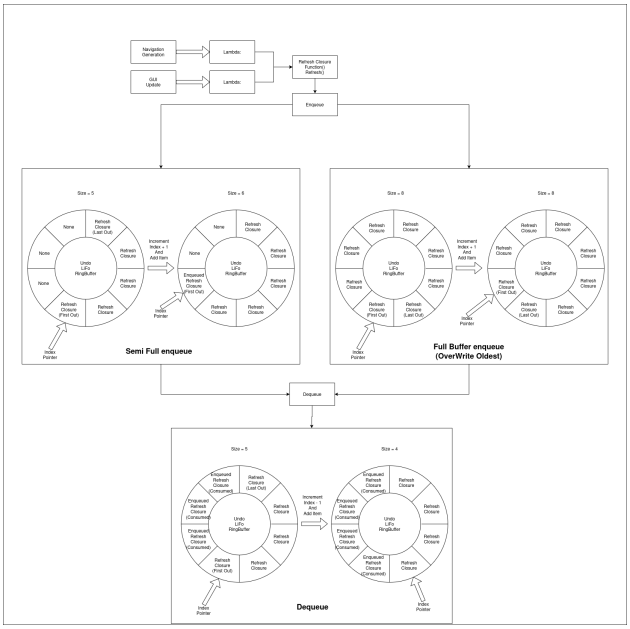

Figure 5: LIFO ring buffer undo structure.

Design Decision: LIFO Ring Buffer

LIFO ordering is correct for undo: the most recently performed action must always be the first reversed. The ring buffer bounds memory usage — once full, the oldest entry is silently overwritten, capping undo history at a configurable depth set in `config.py`. The stored callables contain enough state to restore the previous route view; navigation history can be read from the buffer before it is consumed.

Alternative considered — unbounded stack (`list` with `append/pop`). A plain Python list supports unlimited undo depth but allows unbounded memory growth over a long session. The ring buffer enforces a known, fixed memory footprint.

Complexity Analysis

Operation	Best	Average	Worst
Push action to undo buffer	$O(1)$	$O(1)$	$O(1)$
`undo()` pop and execute	$O(1)$	$O(1)$	$O(1)$

Room and Event Booking System (2.3, 2.5)

*Fulfils requirement: **2.3** (Room and Event Booking System) and **2.5** (Fast Lookup).*

Overview

The booking system uses a two-tier architecture. When a booking is created it is inserted into both the AVL tree archive (Section 7) and the active `LifoRingBuffer`, which holds a reference to the same object for fast $O(1)$ access during the current rolling window. Once a day rotates out of the ring buffer the reference is dropped, but the booking persists in the AVL archive and remains retrievable in $O(\log n)$ by booker name.

Class Hierarchy and Design Decisions

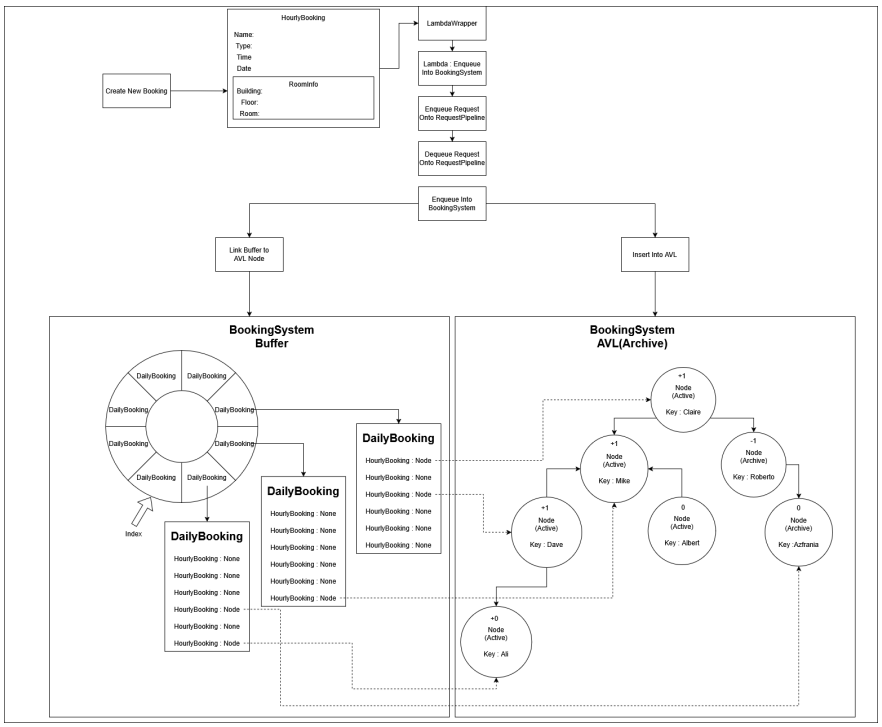

Figure 6: Booking system class hierarchy.

DailyBooking — Flat List of Time Slots

Each `DailyBooking` holds a static Python list of `HourlyBooking` objects, one per time slot:

```python
class DailyBooking:
    def __init__(self):
        self.hourly_bookings = []
        for i in range(len(times) - 1):
            self.hourly_bookings.append(
                HourlyBooking(times[i], times[i+1], None)
            )
```

Listing 6: DailyBooking initialisation

Why a static list? Time slots are fixed and uniformly spaced; the dominant operations are booking or cancelling by slot index ($O(1)$) and slicing a time range ($O(k)$). Because no slot is ever inserted or deleted,therfore static allocation is appropriate for the active window.

Alternative considered — pure AVL implementation. An AVL tree gives $O(\log n)$ insert, delete, and lookup at the cost of higher constant factors and no natural range-slice syntax. The dual-structure approach achieves $O(1)$ for the dominant active-window operations while delegating archived data to the AVL tree described in Section 7.

```
return daily_bookings.hourly_bookings[start_time:end_time]
```

Listing 7: Range query via list slicing

BookingSystem — LIFO Ring Buffer as a Rolling Day Window

Each room's `BookingSystem` stores `DailyBooking` objects in a `LifoRingBuffer` with a configurable capacity (30 days by default):

```
class BookingSystem:
    def __init__(self, capacity):
        self.daily_bookings = lifo.LifoRingBuffer(capacity,
           DailyBooking)
        self.capacity = capacity

    def get_daily_booking(self, index):
        return self.daily_bookings.access_at_index(index)
```

Listing 8: BookingSystem using a ring buffer

When the buffer is full and a new day is pushed, the oldest `DailyBooking` is evicted and its non-empty slots are inserted into the AVL archive before being discarded, ensuring no booking data is lost.

Complexity Analysis

Operation	Best	Average	Worst
`book_room()` / `update_booking()`	$O(1)$	$O(1)$	$O(1)$
`delete_booking()`	$O(1)$	$O(1)$	$O(1)$
`get_daily_booking()` by index	$O(1)$	$O(1)$	$O(1)$
Range query (slot slice)	$O(k)$	$O(k)$	$O(k)$
Name lookup (active window)	$O(C)$	$O(C)$	$O(C)$
Name lookup (archive)	$O(\log n)$	$O(\log n)$	$O(\log n)$
`helper_build_times()` startup	$O(S)$	$O(S)$	$O(S)$

k = slots in the queried range; S = slots per day; C = ring buffer capacity (constant); n = archived bookings.

Space complexity. $O(C \times S)$ for the ring buffer plus $O(n)$ for the AVL archive. Both are bounded: the ring buffer by its fixed capacity, and the archive grows only with historical data.

Summary

The booking system combines a ring buffer for $O(1)$ active-window access with an AVL archive for $O(\log n)$ historical lookup. Together they cover the full booking lifecycle — creation, cancellation, time-range retrieval, and long-term search — without unbounded memory growth.

Incoming Request Processing Pipeline (2.6)

*Fulfils requirement: **2.6** (Incoming Request Processing).*

Overview

The request pipeline is implemented in `RequestPipeline/request_pipeline.py`. It provides a FIFO queue for all GUI requests, ensuring they are handled strictly in arrival order. Each request is wrapped in a `request_lambda_wrapper` before being enqueued.

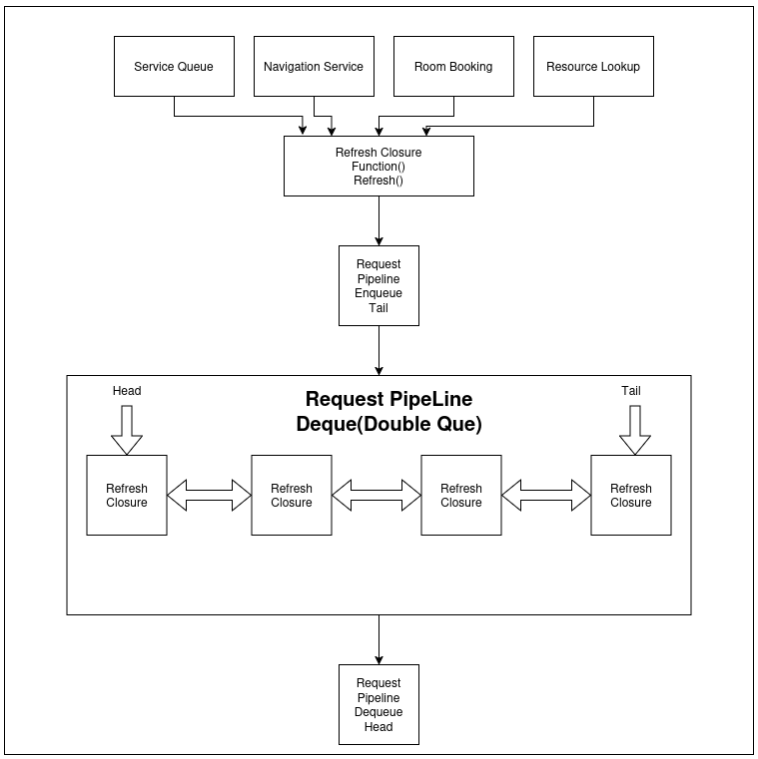

Figure 7: Request Pipeline visualisation.

Request Wrapper

```
class request_lambda_wrapper:
    def __init__(self, position, function, refresh, request_data):
        self.position     = position      # arrival order index
        self.function     = function      # the action to execute
        self.refresh      = refresh       # UI/state refresh
            callback
        self.request_data = request_data  # metadata about the
            request
```

Listing 9: Request wrapper class

The `position` field provides an auditable sequence number for every processed request.

Design Decision: Deque as the Underlying Structure

Requests are added at the tail via `append_tail()` and consumed from the head via `pop_head()`, giving strict FIFO ordering.

Why a deque rather than a plain list? A deque provides $O(1)$ tail insertion and $O(1)$ head removal. A Python list used as a queue requires $O(n)$ for every `pop(0)` call due to element shifting.

Request Execution Model

When dequeued, `function` executes the primary action and `refresh()` updates dependent UI state. Following the command pattern, the pipeline is agnostic to what each request does, keeping it generic and reusable.

Complexity Analysis

Operation	Best	Average	Worst
`enque_request()`	$O(1)$	$O(1)$	$O(1)$
`deque_request()`	$O(1)$	$O(1)$	$O(1)$

Space complexity. $O(n)$ where n is the number of pending requests.

Summary

The request pipeline enforces strict FIFO ordering with $O(1)$ enqueue and dequeue. The command pattern separates execution from pipeline mechanics, keeping the pipeline generic across all request types.

AVL Tree — Balanced Event Index (2.5, 2.7)

*Fulfils requirements: **2.5** (Fast Building and Resource Lookup) and **2.7** (Bonus — Balanced Event Index).*

Overview

The AVL tree is implemented in `DataStructures/AVL.py` and serves two roles: fast keyed building lookup and the long-term booking archive described in Section 5. It guarantees $O(\log n)$ worst-case insert, delete, and search by enforcing height balance after every modification via single (LL/RR) and double (LR/RL) rotations, each running in $O(1)$.

Complexity Analysis

Operation	Best	Average	Worst
`insert()`	$O(\log n)$	$O(\log n)$	$O(\log n)$
`delete()`	$O(\log n)$	$O(\log n)$	$O(\log n)$
`search()`	$O(\log n)$	$O(\log n)$	$O(\log n)$
Rotation (single/double)	$O(1)$	$O(1)$	$O(1)$

Space complexity. $O(n)$ for tree nodes. Each node carries three extra AVL metadata fields (`_avl_height`, `_avl_left`, `_avl_right`), adding $O(1)$ overhead per node.

Summary

The AVL tree guarantees $O(\log n)$ worst-case performance for all keyed operations regardless of insertion order, making it suitable for adversarial patterns such as fully chronological booking insertion. It acts as the archive tier for the booking system and provides worst-case bounded lookup for building resources.

Contributions

Task	**Contributor**
Campus Data Structure (Campus, Building, Floor, Room classes)	Dave
Campus Map & Graph Traversal	Dave
Fast Building Lookup & Booking	Treyson
Room & Event Booking System (`room_booking.py`)	Treyson
Incoming Request Processing Pipeline	Natnael
Priority Service Pipeline (three-tier deque routing)	Tharindu
AVL Tree — Balanced Event Index	Ashar
GUI	Ashar
GUI	Afnan
GUI	Tharindu
Demo Visualizations	Afnan
Report Writing & Documentation	Team

Table 1: Team Contributions

Ai Use Declaration

Model Used : Claude Sonnet 4.6